

Homework 06

Question 1: Compare KNN and logistic regression

Use [breast-cancer.csv](#), which contains 569 cases from the [Wisconsin Diagnostic Breast Cancer dataset](#). Predict y (1 for malignant and 0 for benign) using `mean_radius`, `mean_texture`, and `mean_smoothness`.

Set a random seed and randomly split the observations into 70% training and 30% test data. Use the same split for every model. Standardize the predictors using the training means and standard deviations, and apply the same transformations to the test data.

Fit ordinary logistic regression with an intercept and no penalty, and KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{5, 15, 25, 35, 45\}.$$

Using the estimated probabilities for $y = 1$ on the test data, plot all six ROC curves in one figure and report their AUC values. Which of the six fitted models do you prefer? Give a brief justification based on your results.

Answer

```
Logistic regression AUC: 0.9759053738317758
KNN (k=5) AUC: 0.9494742990654206
KNN (k=15) AUC: 0.9724737149532711
KNN (k=25) AUC: 0.9672897196261683
KNN (k=35) AUC: 0.9718896028037384
KNN (k=45) AUC: 0.9766355140186916
```

With seed 432, the data were split into 398 training observations and 171 test observations, with 37.4% of the test cases being malignant.

The test AUC for logistic regression was 0.9759. The AUC values for KNN with $k = 5, 15, 25, 35$, and 45 were 0.9495, 0.9725, 0.9673, 0.9719, and 0.9766, respectively.

I prefer KNN with $k = 45$ because it achieved the highest test AUC of 0.9766, indicating the best overall ability to distinguish malignant from benign cases among the six fitted models. However, its performance is very similar to logistic regression, which achieved an AUC of 0.9759.

Question 2: Training and test ROC curves

Use the same data, training/test split, and standardization as in Question 1. Fit KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{1, 3, 5, 10, 20\}.$$

Use each fitted model to estimate probabilities for both the training and test observations. For training predictions, include each observation itself among its k nearest neighbors.

For each k , use a separate panel to show the training and test ROC curves together. Report both AUC values.

Explain the difference between training and test performance and how it changes with k . In particular, explain what happens at $k = 1$ and why training performance may not reflect performance on new observations.

Answer

```
Training and Test AUCs
k = 1: Train AUC = 1.0000, Test AUC = 0.9032
k = 3: Train AUC = 0.9890, Test AUC = 0.9468
k = 5: Train AUC = 0.9868, Test AUC = 0.9495
k = 10: Train AUC = 0.9826, Test AUC = 0.9623
k = 20: Train AUC = 0.9846, Test AUC = 0.9679
```

ROC Curves: Logistic Regression vs. KNN

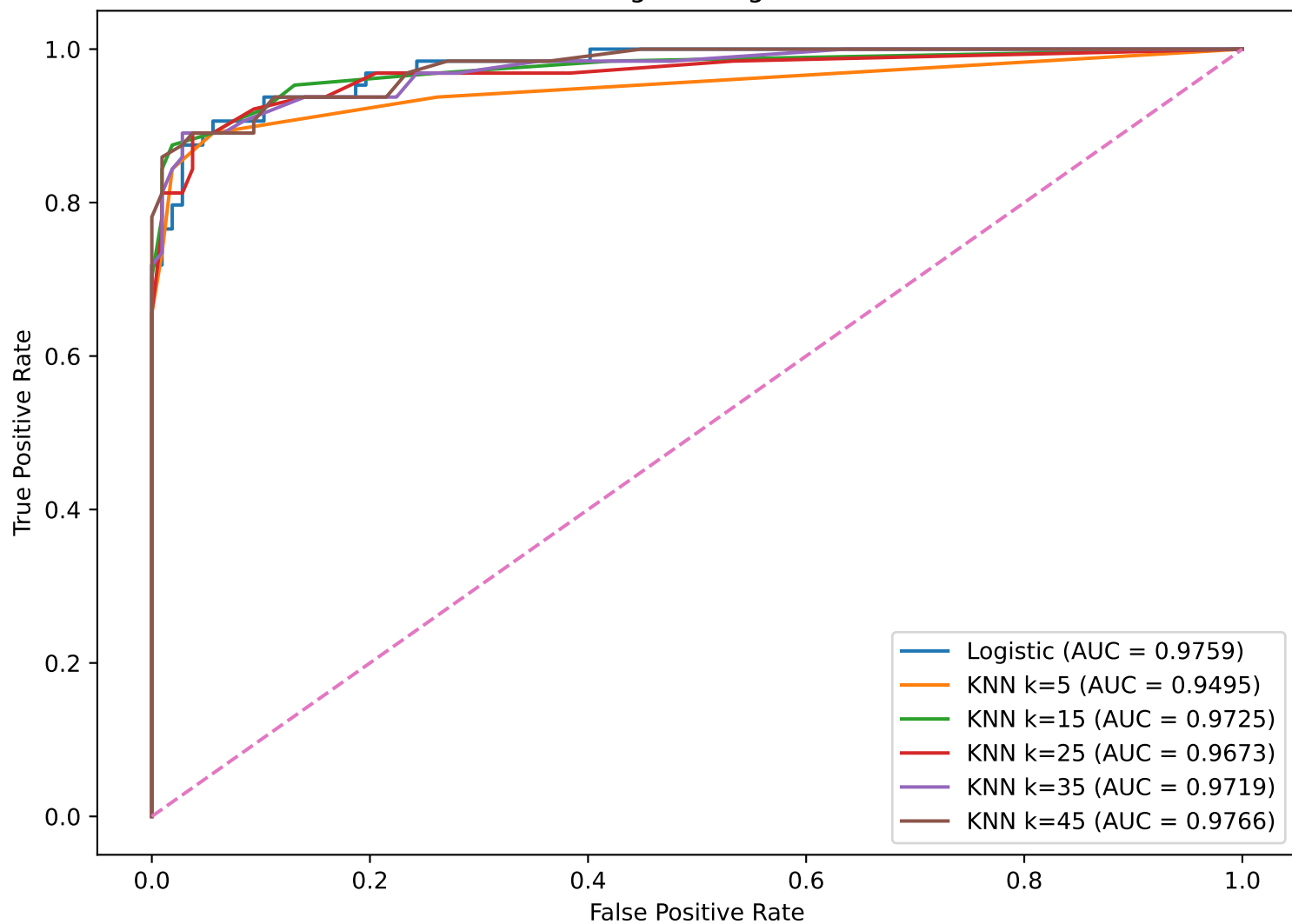


Figure 1: Test-set ROC curves for logistic regression and KNN ($k = 5, 15, 25, 35, 45$).

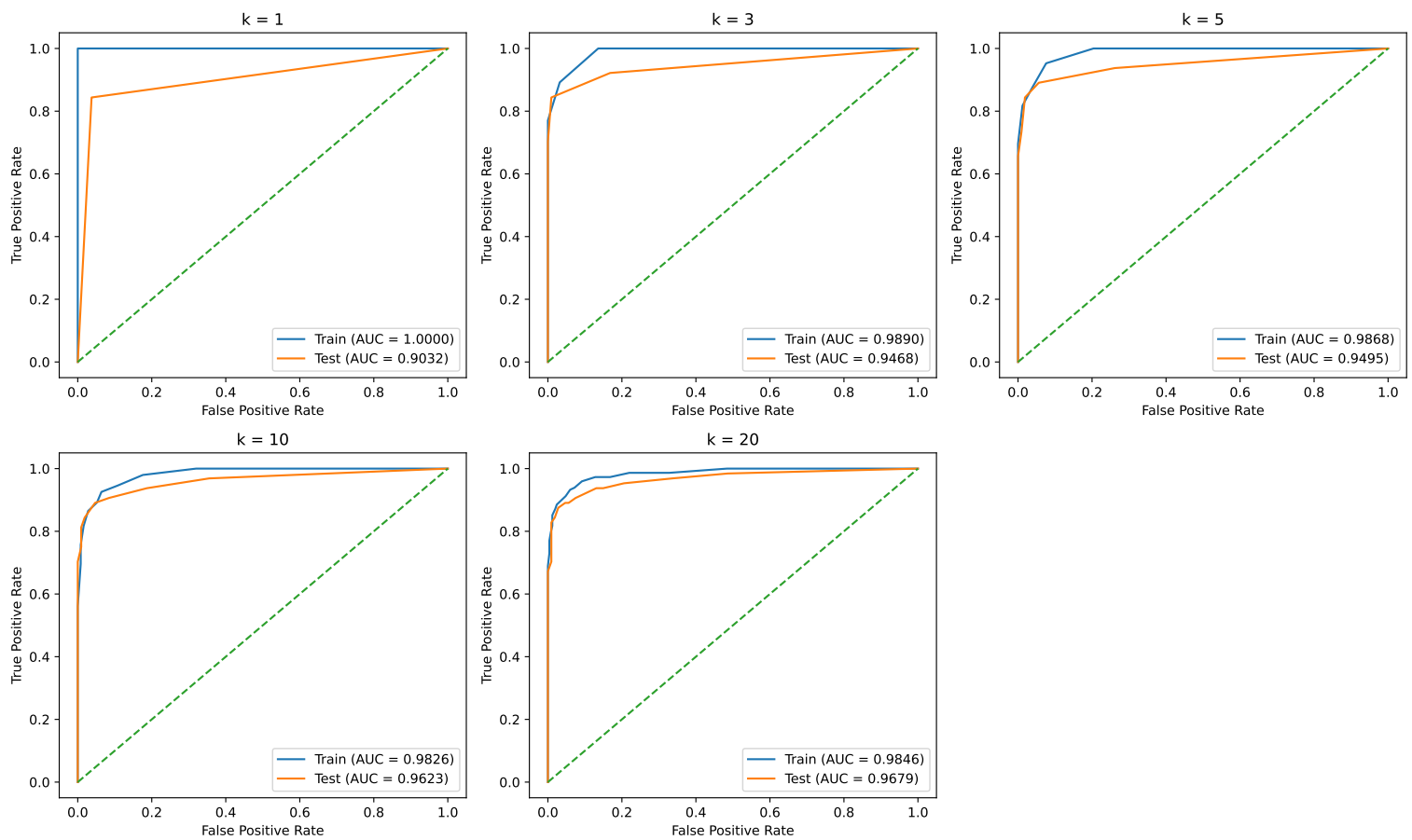


Figure 2: Training and test ROC curves for KNN with $k = 1, 3, 5, 10, 20$.

Training performance was higher than test performance for every value of k , but the gap between them decreased as k increased. The difference between training and test AUC was 0.0968 at $k = 1$, 0.0422 at $k = 3$, 0.0373 at $k = 5$, 0.0203 at $k = 10$, and 0.0167 at $k = 20$. Thus, increasing k substantially reduced the difference between training and test performance.

At $k = 1$, the training AUC was 1.0000 because each training observation was included among its own nearest neighbors. Its closest neighbor was therefore itself, so the model perfectly predicted the training observations. However, the test AUC was only 0.9032 because new observations cannot use themselves as neighbors.

As k increased, predictions were based on more neighboring observations. This made the model less sensitive to individual training points and reduced overfitting. The training AUC generally decreased slightly (from 1.0000 at $k = 1$ to about 0.98), while the test AUC increased from 0.9032 at $k = 1$ to 0.9679 at $k = 20$. Therefore, the training-test AUC gap became progressively smaller, indicating better generalization to new observations. This also shows why training performance alone can be misleading: a model can perform extremely well on data it has already seen while performing substantially worse on new data.

Question 3: Peer evaluation of Homework 5

You will receive two Homework 5 submissions to review. The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 5 submission. It will contain links to two other students' repositories, where you can find their submitted homework reports.

Open your repository's **Issues** tab on GitHub to find the assignment. You may ask AI to explain GitHub Issues or help you locate the assignment.

Choose **one specific mistake or item for improvement** from any question in either report.

1. **Identify the mistake or item for improvement.** State which report and question you selected, and briefly describe what is wrong or could be improved. Examples include incorrect code, a calculation error, a misinterpretation of results, a plot that could communicate the findings more clearly, or anything else you think could be done better.
2. **Explain your assessment.** Quote the relevant text, code, or figure caption exactly. Explain why the selected answer is incorrect or how it could be improved, referring to its reasoning, calculations, code, figures, or interpretation. **Complete this part yourself, without AI assistance.**
3. **Provide a correction.** Write a corrected or improved answer to the selected question, including any calculations, code, or figures needed.

After completing your review, **open a GitHub Issue in the repository containing the report you selected.** Identify the report and question, and include your assessment and a detailed revision plan explaining what the author should change and why. Include a link to your posted GitHub Issue in your Homework 6 answer.

Answer

1. Issue

I selected **nickpanico's** Homework 5 report, Question 1. The overall bias-variance interpretation is correct, but saying the MSE is lowest “somewhere in the middle” is too vague. The report already calculates the MSE for every value of k , so it should identify the exact value of k that gives the smallest estimated MSE. From the reported results, the minimum MSE is 0.0120464 at $k = 9$.

2. Why it could be better

The report states:

“This creates a bias-variance tradeoff, which is why the MSE is lowest somewhere in the middle.”

The purpose of calculating MSE across the different values of k is to examine how the bias-variance tradeoff affects prediction error. Reporting the exact minimum makes the interpretation more precise and directly connects the explanation to the simulation results. Instead of only saying that the best value is “somewhere in the middle,” the response can show that $k = 9$ provided the best balance between squared bias and variance among the values considered.

3. Improved answer

The estimated MSE was approximately equal to squared bias plus variance for each value of k , confirming the bias-variance decomposition using the 200 simulated training sets. As k increases, variance generally decreases because the prediction is averaged over more neighboring observations. At the same time, squared bias generally increases because larger neighborhoods include observations farther from x_0 .

This creates the expected bias-variance tradeoff. In this simulation, the MSE reaches its minimum value of 0.0120464 at $k = 9$, so $k = 9$ gives the best observed balance between bias and variance among the values considered. This is similar to ridge regression or lasso, where increasing regularization makes the model less flexible, generally reducing variance while increasing bias.

The code should also identify the minimizing value of k directly instead of leaving it to be read off the table. The original code only prints the MSE for every k :

```
mse <- colMeans((predictions - f_x0)^2)

data.frame(
  k = k_values,
  mse = mse
)
```

Keeping that table, the report should add:

```
best_index <- which.min(mse)

data.frame(
  k = k_values[best_index],
  mse = mse[best_index]
)
```

Using the report's MSE values, this returns $k = 9$ with $\text{MSE} = 0.0120464$.

GitHub Issue

Posted peer review: <https://github.com/nickpanico/stat432-fall2026/issues/2>